VerifierShift: An Authoritative-Evidence Intervention Across Three Task Schemas

SAURAV K. ARYAL, Howard University, USA

A task verifier cannot reliably infer an outcome when its observations omit a policy that determines correctness. We study this boundary using three authored SQLite workflow schemas: room booking, multi-item inventory dispatch, and support escalation. Twenty-four paired constructions produce 48 worlds with opposite SQL-derived outcomes but identical incomplete evidence within each pair. A transaction-consistent authoritative read then adds the current policy and required state. A typed checker resolves 48/48 complete-evidence cases correctly and reviews all 24 incomplete observation classes and 12 invalid-source controls. Two local model judges make 168 actual calls on the same 84 unique packets per model. On complete evidence, Gemma 4 (26B) produces 34/48 correct determinate decisions with 7 false acceptances; Qwen 3.5 (4B) produces 15/48 with 14 false acceptances and 17 reviews. Three invalid responses remain in the record. Both models review all incomplete classes; a validity gate corrects two unsupported Qwen acceptances on invalid-source packets. The pilot supports explicit evidence contracts and a strong typed baseline. It does not establish calibrated learned autonomy, public-simulator transfer, or assurance for untrusted capture. Future work connects this bounded intervention to versioned agent trajectories and evaluates whether model reasoning adds useful coverage beyond executable checks.

Additional Key Words and Phrases: agent verification, policy shifts, tool evidence, evidence contracts, selective review

1 Introduction and Background

An agent can report success while leaving the wrong committed state. A verifier may inspect that state and still draw the wrong conclusion if the governing policy is unavailable, stale, or attached to a different entity. Distinguishing a failed task from insufficient evidence is therefore a prerequisite for deciding whether to accept an outcome or request review. A model's confidence cannot supply a policy observation that was never recorded.

Stateful evaluation is established prior work. The τ-bench benchmark compares final database state with annotated goals in policy-constrained tool interactions [8]. ToolSandbox includes state dependencies and intermediate evaluation milestones [3]. AgentRewardBench studies automatic judges against expert trajectory annotations and reports variation across benchmark settings [4]. DynSTEER investigates dynamic stage-wise evaluation and execution-time review [6]. These systems motivate evaluation at explicit execution boundaries; this pilot does not reproduce them or introduce state-based evaluation as a new idea.

Our question is narrower: what changes when a verifier obtains an authoritative observation that its original evidence lacked? We construct paired worlds that share an incomplete evidence packet but require incompatible task decisions. We then obtain a real, transaction-consistent database snapshot and compare deterministic checks with two model judges. The intervention changes the available information. Any resulting improvement is consequently distinct from an algorithm outperforming another algorithm on equal observations.

The contribution is an auditable multi-schema feasibility artifact and its failure accounting. The schemas require different relations: overlapping booking intervals and capacity, multi-SKU quantity conservation with unit conversion, and entitlement, queue, and deadline constraints. The contracts are authored for these schemas; they are not discovered by a learned verifier. The study expands beyond a single refund-policy template while remaining a small constructed benchmark. The artifact is available at https://github.com/Saurav-K-Aryal/verifiershift-pilot.

Author's Contact Information: Saurav K. Aryal, Howard University, Washington, DC, USA.

2 Methods

2.1 Worlds, policies, and independent labels

The frozen generator creates eight logical pairs for each of three families. Each pair contains two separately instantiated SQLite databases. The visible committed effects are held fixed within the pair, while an authoritative policy changes so that one world satisfies the task and the other does not. There are 24 logical pairs and 48 worlds, not 48 independent workflows. Exactly 24 complete worlds are correct by construction; that balanced prevalence is not an operational estimate.

For booking, eligibility depends on requested seats and the total seats in other bookings that overlap a requested interval. The policy supplies capacity and a temporal buffer. An eligible request requires one exact booking; an ineligible request requires none. Policy variants alter capacity or whether a nearby interval counts as overlapping. The SQL evaluator uses interval comparisons and aggregate occupancy, rather than a textual success message.

Inventory dispatch converts package requests into units using the current policy. Every requested SKU must remain above a reserve floor. Eligible dispatch requires one exact shipment per SKU and matching stock decrements; ineligible dispatch requires no shipment and unchanged stock. Policy variants change the reserve floor or units per package. Support escalation instead combines customer eligibility with required queue and SLA deadline. Policy variants alter minimum tier, queue, or the allowed delay. These distinct state relations are the intended schema-transfer diagnostic.

Task labels come from SQL queries against each database. The public checker consumes JSON snapshots and implements the stated predicates separately. A second, independently written SQL readback recomputes all 48 labels. This separation can expose implementation disagreement on the constructed cases; it does not provide an independently elicited human specification or prove the contracts complete for real services.

2.2 Evidence channels and the capture boundary

Each incomplete packet contains the actor-visible cached policy and committed-state description but lacks an authoritative current-policy read. Within each logical pair, the two serialized incomplete packets are byte-identical and their SQL labels differ. A single packet therefore cannot justify opposite determinate verdicts. We require review for this observation class and make one model call per class, avoiding duplicate calls on identical bytes.

The complete channel performs an actual SQLite read transaction that captures the current policy and required state. The packet binds policy and state sources to the requested entity, schema version, capture epoch, and their respective revision identifiers. SQLite provides the transaction machinery used for this capture [7]; distributed consistency and concurrent capture failures are outside the experiment. Policy revision values are random identifiers rather than encoded world labels.

Twelve invalid-source controls, four per family, remove a required policy source, supply a mismatched revision, duplicate a conflicting policy source, or bind a source to another entity. These controls require review. The validator checks required source presence and uniqueness, entity and metadata links, and stored capture digests. The trusted local adapter supplies these records. A digest does not authenticate an arbitrary source that an adversary can forge, and a schema-version field is not a general malformed-payload verifier. Complete, honest capture is an explicit assumption.

Table 1. Unique native inputs per model. Incomplete classes each correspond to two incompatible worlds; they are called once.

Channel	Packets	Required decision
Incomplete current authority	24	Review
Complete authoritative read	48	Accept 24; reject 24
Invalid-source controls	12	Review

2.3 Checkers, models, and frozen execution

The primary baseline is the typed evidence-and-postcondition checker. A validity-only gate is also applied to each model verdict: when the source contract is invalid, it returns review; otherwise it retains the model's decision. This gate does not replace a model's incorrect interpretation of a valid complete packet with the typed postcondition result.

The model handles are `gemma4:26b` and `qwen3.5:4b`. Official family documentation is cited for context [2, 5]; the actual local model digests and backend records identify the executed configurations. Gemma uses Q4_K_M through llama.cpp; Qwen uses an NVFP4 MLX backend. Capacity, architecture, post-training, precision, and execution implementation differ. Their descriptive comparison does not isolate a model-family or release-age effect.

Each model receives the same frozen evidence, a workflow-specific written contract, and a fixed instruction to return a JSON verdict of accept, reject, or review. Private labels, latent condition names, and source filenames are excluded from requests. Temperature is 0, the context limit is 4,096 tokens, the output cap is 160 tokens, and the native thinking option is false. Confidence is not requested. There is no prompt, threshold, or model selection after responses are observed.

The finite budget is $2(24 + 48 + 12) = 168$ native calls. Models run serially and are unloaded after each block. A CPU-only metadata amendment randomized revision identifiers, froze model digests, and clarified residence records before any native response; superseded CPU artifacts remain preserved. The final source, packet manifest, request order, and options are frozen before inference. Invalid or truncated responses are retained without repair or retry.

3 Results

3.1 Complete evidence resolves the typed predicates

The typed checker returns the correct determinate verdict on all 48 complete packets, including all 24 correct and 24 incorrect worlds. It also reviews all 24 incomplete classes and all 12 invalid-source controls. Each family contributes 16 complete worlds. These results establish consistency among the authored contract, captured JSON, and independently checked SQL labels on this fixture. They do not estimate a population failure rate or certify a universal checker.

Gemma produces 34/48 correct determinate decisions on complete evidence:17 correct accepts and 17 correct rejects. It also produces 7 false accepts,6 false rejects, and 1 invalid output. Qwen produces 15 correct accepts,14 false accepts,17 reviews, and 2 invalid outputs; it produces no rejects in this channel. Review is recorded separately from an unsafe acceptance. Figure 1 preserves those distinctions instead of collapsing them into a single accuracy or autonomy score.

The model results vary by schema (Figure 2). Gemma obtains 8/16 correct determinate booking decisions,12/16 inventory decisions, and 14/16 escalation decisions. Qwen obtains 1/16,7/16, and 7/16, respectively. Booking combines interval and aggregate conditions; inventory includes both conversion and conservation. These small constructed counts identify cases worth inspecting, rather than establishing a general hierarchy of task difficulty.

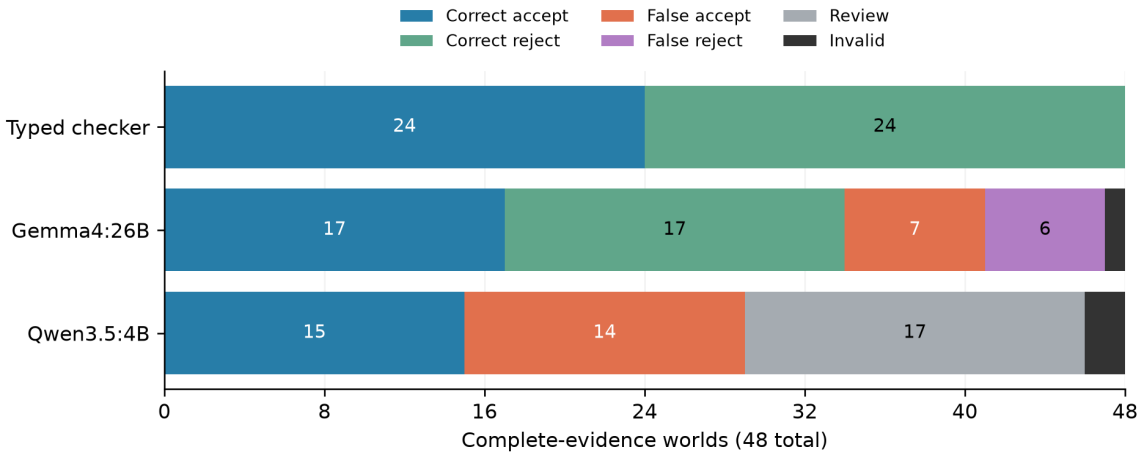


Fig. 1. Observed complete-evidence outcomes. Every row contains the same 48 constructed worlds, balanced 24 correct/24 incorrect. Review and invalid outputs remain separate from false acceptance.

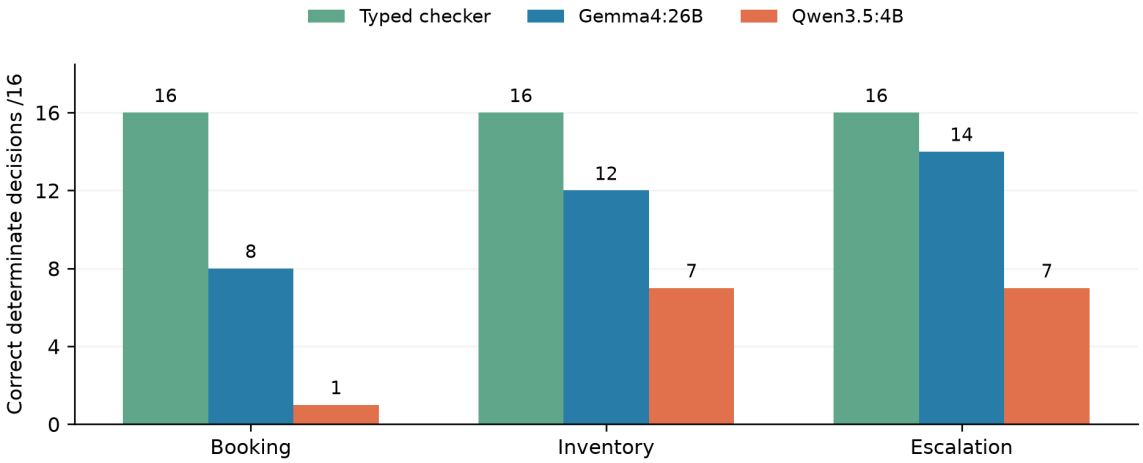


Fig. 2. Correct determinate decisions on 16 complete worlds per authored schema. The typed contract is explicitly implemented for each family; its result is not zero-shot learned schema induction.

3.2 Information limits and invalid-source controls

Both models review all 24 incomplete observation classes. Each such response applies to a pair of worlds with opposite truth, so reporting 48 reviews as 48 independent native calls would be incorrect. The experiment directly verifies the relevant ambiguity: the input bytes match while the independent SQL outcomes differ. The complete read removes this designed ambiguity for the typed checker, although it does not make model interpretation reliable.

Gemma reviews all 12 invalid-source packets. Qwen reviews 10 and accepts 2 without a valid source contract. Because those packets lack qualified evidence, we describe these as unsupported determinate decisions rather than assuming their underlying task outcomes are necessarily false. The validity-only gate changes both to review and leaves

the complete-evidence model results unchanged. This is a bounded demonstration of a prerequisite gate, not a new authorization or transaction protocol.

The independent readback checks all 48 SQLite worlds and all 168 raw native requests, their model identities and options, parsed verdicts, and aggregate counts. One Gemma response and two Qwen responses hit the output limit and fail the required verdict schema; they remain invalid. No new calls were made to replace them. The public artifact retains raw responses and provides CPU verification without distributing model weights or database binaries.

3.3 Interpretation and limits

The substantive result is that explicit authority changes what can be established, while a complete source packet still leaves model errors. The typed checker is a strong comparator precisely because every tested postcondition is executable. Adding a model to these already decidable cases has not shown useful extra coverage. The pilot does not measure learned confidence calibration. Formal risk-control methods address different statistical questions under stated conditions [1]; they do not turn missing policy information into an observation.

The fixtures are hand-authored and intentionally balanced. Related worlds share parameters and effects, and all three contracts were written for this study. The experiment does not contain naturally occurring agent trajectories, public-simulator tasks, human adjudication, adversarial capture, or distributed failures. Metadata controls test declared source prerequisites; arbitrary malformed payloads and forged authorities are not covered. The evidence intervention and algorithm comparison must remain separate analyses in any extension.

4 Future Work

The immediate extension is a versioned change layer over licensed public agent simulators, preserving their upstream task evaluators. A first quarter should reproduce two simulator adapters and audit about 100 episodes before scaling. The proposed year-one target is 600 base instances divided into development, calibration, and held-out groups, with linked variants and close paraphrases kept together. A separate workflow-family transfer test should measure how much contract authoring and adapter work is needed, rather than treating the present three schemas as broad generalization.

The evidence contract should record source identity, policy and tool versions, units, timestamps, and the capture boundary. Prespecified changes should include published policy revisions, altered API field semantics, delayed effects, missing records, and inconsistent source versions. Complete and incomplete information channels must remain explicit. A Strands-compatible recorder is a practical integration target; real accounts and live transactions are not needed to study the initial benchmark.

Typed predicates should remain the primary baseline. A model may be useful for genuinely semantic requirements, cited interpretation, or identifying an unresolved source, but it should not override a failed hard predicate or invent missing evidence. Compare such additions on the same captured trajectories and available observations. Report false accepts, correct-case acceptance, unresolved cases, and measured cost per correctly resolved episode. Exclude a component from the recommended route if it adds cost without independently checked utility.

The later quarters should freeze an untouched change manifest and operating rules, run bounded cross-model comparisons, and release failures as well as successes. Human reviewers should adjudicate a prespecified subset of semantic labels. Cost and coverage claims need their own measured routing experiment: this pilot made all 168 planned model calls and does not claim inference time saved by hypothetical gate-first execution. The intended deliverables are an executable contract library, recorded-source adapter, benchmark manifest, and cost report suitable for a larger assurance study after one year.

5 Conclusion

Across three authored SQLite task schemas, identical incomplete evidence corresponds to incompatible outcomes. An actual authoritative read supplies the missing information for 48/48 correct typed decisions, while the two model judges retain substantial complete-evidence errors. Required-source checks also prevent two unsupported model acceptances. These observations support a versioned evidence-contract benchmark with explicit information boundaries and a strong executable baseline. Useful learned autonomy and transfer to public agent workflows remain questions for the proposed follow-up.

Artifact preparation. Saurav K. Aryal directed the preliminary experiments, with Codex assistance for implementation, execution, analysis, and manuscript preparation. Public manuscript sources are distributed inside the repository's `overleaf.zip`.

References

[1] Anastasios Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/f3549ef9b5ff520a7e41ff3cc306ab2b-Abstract-Conference.html

[2] Google DeepMind. 2026. Gemma 4 Model Card. Official model-family documentation. https://ai.google.dev/gemma/docs/core/model_card_4 Accessed October 31, 2026; local execution digest and backend are reported separately.

[3] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 1160–1183. doi:10.18653/v1/2025.findings-naacl.65

[4] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv:2504.08942 https://arxiv.org/abs/2504.08942v2 Version 2, revised October 6, 2025.

[5] Qwen Team. 2026. Qwen3.5-4B Model Card. Official model documentation. https://huggingface.co/Qwen/Qwen3.5-4B Accessed October 31, 2026; local quantized backend identity is reported separately.

[6] Zhichao Shi, Xuhui Jiang, Wenjie Zhang, Xiaojun Wu, Cehao Yang, Chengjin Xu, Jian Guo, and Yuanzhuo Wang. 2026. DynSTEER: Dynamic Stage-wise Trajectory Evaluation and Execution-time Review for Agents. arXiv:2609.14637 https://arxiv.org/abs/2609.14637v3 Preprint, version 3; revised September 26, 2026.

[7] SQLite. [n. d.]. Isolation In SQLite. Official documentation. https://sqlite.org/isolation.html Accessed October 31, 2026.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html